

7.2.23

## Computation

It can be defined as finding a solution to a problem from the given inputs by means of algorithm.

IPO (Input Process Output)

FSM (Finite State Machine)

→ NFA :- Non Deterministic <sup>Finite</sup> Automata

→ DFA :- Deterministic <sup>Finite</sup> Automata

• FSM is a model of computation consisting of set of states, a start state, input alphabet, and transition function that maps input symbols and current states to a next state.

• Computation begins in the start state with an input string. It changes to next state depending upon the transition function.

(Transition function suggest how one state is converted to next state)

- It is an automaton if it is a self operating machine.
- An automaton is a system which obtains, transforms transmits and uses information to perform its function w/o direct human participation. It is self-operational.

10.2.23

Components of FSA (Finite State Automata) :-

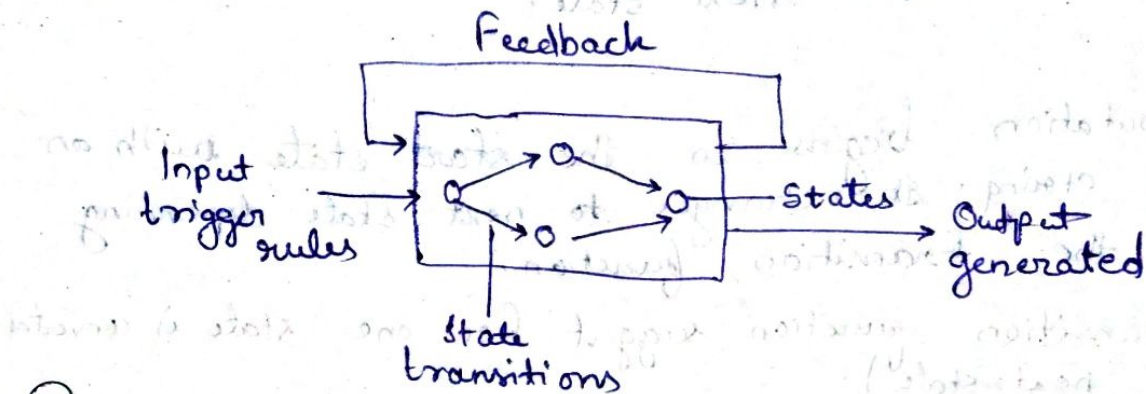
- Control Unit (Transition diagram for logic)
- read unit (read one character at a time)
- input tape (long tape, input alphabets and characters are stored in it)



## Elements of FSN

There are four main elements :-

- (i) State which defines the behaviour & may produce action.
- (ii) State transition which are movements from one state to another.
- (iii) Rules or conditions which must be met to allow a state transition.
- (iv) Input events which are either internally or externally generated. These may trigger these rules & lead to state transitions.



## Representation of FSA

- ① State diagram
- ② State transition table
- ③ Transition function

start



- final or  
accepting  
state

→ transition

## State :-

It is a complete set of properties, transmitted by an object to an observer through one or more channel.

## Types :-

- (i) Start State :- Initial state/condition of FSM
- (ii) Accepting State :- If a FSM finishes an input string and is in an accepting state, the string is accepted and is considered to be valid.

State immediately following the current state.

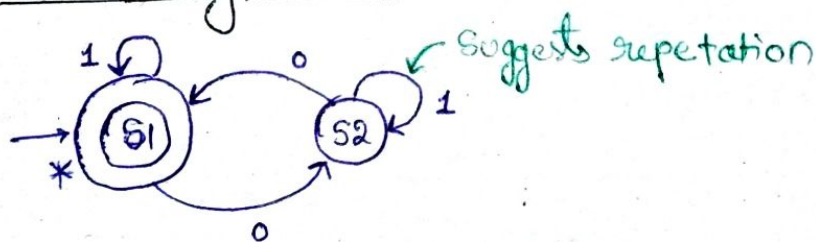
- iii) Universal state :- This state only accepts all possible moves leading to acceptance is called universal state.

- iv) Existential state :- Which accept any moves that leads to acceptance is called existential state.

- v) Dead and trap state :- Non-final state whose transition on every input symbols terminate on itself.

Transition :- It is the act of passing from one state to the next state then the FSM (Finite State Machine)

## State Diagram



- Final state is mentioned with '\*' at least one.
- Transition must contain at least one final state
- For start state ( $\rightarrow$ ) small arrow is used to represent.

| S \ I                 | Inputs |    |
|-----------------------|--------|----|
|                       | 0      | 1  |
| * S1<br>(Final State) | S2     | S1 |
| S2                    | S1     | S2 |

It is the transition table.



# Finite Automaton

14.2.23

# Finite A finite automata has

(i) a finite set of state, one of which is designated as initial state or start state and some <sup>or</sup> may be more of which are designated as final states.

(ii) an alphabet signal ( $\Sigma$ ) of possible input symbols

(iii) a finite set of transitions that informs each state and each symbol of the input alphabet of the next state.

$$A = \{Q, \overset{\text{sigma}}{\Sigma}, \underset{\text{delta}}{\delta}, q_0, F\} \quad \text{FSM has 5 characteristics}$$

$Q$  is finite non-empty set of labels identifying the states of the machine.

$\Sigma$  is finite vocabulary of input or data symbols.

$\delta$  is the transition function of a machine

$$\delta : Q \times \Sigma \rightarrow Q \quad (Q \times \Sigma \text{ into } Q)$$

$q_0 \in Q$ ,  $q_0$  must be present in  $Q$

$F \subseteq Q$ ,  $F$  (Final State) is subset of  $Q$  or must be within  $Q$

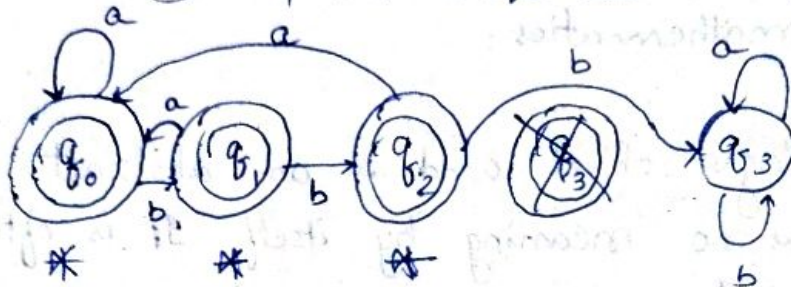


# Language of Automaton

A FSM accepts a string  $n = w_1, w_2, \dots, w_i$  if there is a path in the transition diagram such that

- it begins at a start state
- ends at an accepting state

and has labels  $w_1, w_2, \dots, w_i$



| <del>1/P</del>              | <del>Present</del> |
|-----------------------------|--------------------|
| <del>states</del>           | <del>Inputs</del>  |
| <del><math>q_0</math></del> |                    |

| Inputs | Present | Inputs |
|--------|---------|--------|
| States |         |        |
| $q_0$  | $q_0$   | $q_1$  |
| $q_1$  | $q_0$   | $q_2$  |
| $q_2$  | $q_0$   | $q_2$  |
| $q_3$  | $q_3$   | $q_3$  |

$$\begin{aligned}
 w &= q_0 a b b a \\
 &= (q_0 b) b a \\
 &= (q_1 b) a \\
 &= (q_2 a) = q_0
 \end{aligned}$$

\* If we reach final state after reading each character individually then the string is acceptable.

$$\begin{aligned}
 w &= a b b b a \\
 &= q_0 b b b a \\
 &= \dots
 \end{aligned}$$



# Formal Language Theory

Computer is a symbol manipulator. Language is a system of signs used to communicate information to others.

Language of computation is a combination of both english and mathematics.

Symbol is a single object and is an abstract entity that has no meaning by itself. It is often called uninterpreted.

|                      |                      |                             |
|----------------------|----------------------|-----------------------------|
| $\alpha$ (alpha)     | $\kappa$ (kappa)     | $\chi$                      |
| $\beta$ (beta)       | $\lambda$ (Lambda)   | $\psi$ (Psi)                |
| $\gamma$ (gamma)     | $\mu$ (mu)           | $\omega$ (Omega)            |
| $\delta$ (delta)     | $\nu$ (Nu)           | $\forall$ (forall)          |
| $\epsilon$ (epsilon) | $\omicron$ (Omicron) | $\exists$ (exists)          |
| $\zeta$ (zeta)       | $\pi$ (pi)           | $\propto$ (proportional to) |
| $\eta$ (eta)         | $\Sigma$ (Sigma)     | $\dots$ (ellipsis)          |
| $\theta$ (theta)     | $\Phi$               |                             |

$A \rightarrow B$  (A maps to B)

$\exists$  (there exists)

$\propto$  (proportional to)

$\subset$

$\mathbb{C}$

$\therefore$  (As)

$\dots$  (ellipsis)

$\Sigma = \{0, 1\}$

## Strings

16.2.23

A string is a finite sequence of symbols chosen from some alphabets.

WS usually denoted by string alphabets

UVWXY are strings

ijklmn are natural numbers

## Length of the String

It is defined as the no. of symbols in the string and is denoted by  $|x|$  (length of  $x$ )

A null string is a string with no symbols. It is a string of length 0. It is usually represented by  $\epsilon$

## Operations on String

① Concatenation  $\rightarrow$  It is simply <sup>the</sup> doing of strings together and placing them adjacent to each other in order to form a new string.

Mathematically,

$$S_1, S_2$$
$$S' = S_1 \circ S_2 = \{x \in S_1, y \in S_2\}$$

$\uparrow$  concatenation

$$* x^0 = \epsilon$$

$\uparrow$  means no symbol

②  $* x^{i+1} = x^i \circ x$

Kleen  $\rightarrow$  Stephen Kleen

② Kleen Closure  $\rightarrow$  Kleen star (\*) or Kleen closure is unary operation either on a set of strings or on sets of symbols.

$a^*$  means  $\epsilon, a, aa, \dots$

$\epsilon^*$  is used in regular expression;



③ Positive Closure  $\rightarrow$  It is represented by  $S^+$ .

Any set excluding  $\epsilon$  is positive closure.

Alphabets  $\{a\}$

$$a^* = \epsilon, a, aa, \dots$$

$$a^+ = a^* - \epsilon$$

Ex (i)  $S^* = S^+$

$$S = \{\epsilon, a\}$$

$$S^* = \{\epsilon, a, aa, aaa, \dots\}$$

$$S^+ = \{a, aa, aaa, \dots\}$$

(ii)  $S^* \neq S^+$

$$S = \{a\}$$

$$S^* = \{\epsilon, a, aa, \dots\}$$

$$S^+ = \{a, aa, aaa, \dots\}$$

(iii)  $S = S^*$

$$S = \{\epsilon\}$$

also acceptable

$$S = \{\epsilon, a, aa, \dots\}$$

$$S^* = \{\epsilon, a, aa, \dots\}$$

(iv)  $S \neq S^*$

$$S = \{a\}$$

$$S^* = \{\epsilon, a, aa, \dots\}$$

(v) A set of strings such that  $S^*$  is finite

$$S = \{\epsilon\}$$

$$S^* = \{\epsilon\}$$



- ④ String Reversal  $\rightarrow$  Intuitively when a string  $x$  is reversed. Str A string i.e spelled backwards. ~~It is de~~ Then  $x^R$  is the resulting string which satisfies
- (i)  $x^R(i) = x(n+1-i)$  for  $1 \leq i \leq n$
  - (ii)  $|x^R| = |x|$

### ⑤ Parts of Strings

There are 3 parts of a string

- (i) Leading part  $\rightarrow$  prefix
- (ii) middle part  $\rightarrow$  substring
- (iii) Trailing Part  $\rightarrow$  suffix

Ex)  $*x = yz$

then for some  $z$ ,  $y$  is prefix of  $x$

\* if there exists string  $y$  and  $u$ , say

$$x = yzu$$

then

$z$  is a substring of  $x$

\* If  $x = yz$  for some  $y$  then  $z$  is a suffix of  $x$ .

### Formal Language

# A formal language or simply language is an abstraction for general characteristics of programming language that can be defined as a set of strings all of which are chosen from some particular alphabet.

# A formal language is a set of finite length words that is character strings drawn from some finite alphabet.

\* English can't be represented as formal lang.

\* Prog. languages can be represented as formal language.

# The formal languages can be represented or are represented by

$L(M)$  machine  
 $L(G)$  grammar  
 $L(r)$  regular expression

### Regular Set

A regular set is a set of strings for which there exists some finite automata ~~Accept except that~~ ~~state~~ accepting that set, i.e., if  $R$  is a regular set then  $R = L(M)$  for some finite automata  $M$ . Similarly if  $M$  is a finite automata then  $L(M)$  is always a regular set.

### Formal Language Specification

Formal language can be specified in a no. of ways

(a) A string produced by some formal grammar and this formal grammar is according to Chomsky's Hierarchy

(b) Strings produced by a regular expression

(c) Strings accepted by some automaton such as

Turing Machine (TM)



\* TM is most complex using which all programming languages can be recognized.

\* Context free grammars used in programming syntax checking.

\* Contextual grammars cannot be used in programming language.

④ Exams from a set of related yes/no questions to those for which an answer is yes.

### Category of Formal Languages

① Recursive Language

② Recursively Enumerable Language

↳ a language which gives various statements using nos.

③ Context-Sensitive language

④ Indexed Language

⑤ Deterministic Context-free language

### Operations on Languages

① Union

$$x \in L_1 \cup L_2 \text{ iff } x \in L_1 \text{ or } x \in L_2$$

$$\Sigma^* = L \cup \bar{L}$$

② Complementation :- Operations performed in a single language.

$$x \in \bar{L} \text{ iff } x \in \Sigma^* - L$$

### ③ Concatenation of Language

If two languages are there then

$$L_1 \circ L_2 = L_1 L_2$$

$$\{xoy \mid x \in L_1, \text{ and } y \in L_2\}$$

$$L \circ \phi = \phi \circ L = \phi$$

← null language (contains no string)

$$L^0 = \{\epsilon\}$$

$$L^{i+1} = L^i \circ L$$

### ④ Reversal of Language

$$L^R = \{w^R \mid w \in L\}$$

Ex) If  $L = \{a_1 b_1, a_2 b_2, a_3 b_3, \dots\}$

$$L^R = \{b_1 a_1, b_2 a_2, b_3 a_3, \dots\}$$

### ⑤ Palindrome Language

A language called palindrome over sigma

alphabet  $\rightarrow \Sigma = \{a, b\} \rightarrow$  strings aa, ab, bb etc

Palindrome  $\rightarrow P = \{\epsilon, \text{all strings } w \text{ such that } w^R = w \text{ and } w \in \Sigma\}$

Ex)  $\{\epsilon, a, b, aa, bb, aaa, \dots\}$



## Regular Languages Expression

Regular languages are those that can be constructed from the simple set of alphabets with operations, unions, concatenation and Kleen star.

Every <sup>regular</sup> language can be expressed as an expression or formula by using the set operations of union, concatenation and Kleen star.

### Definition

$\Sigma \rightarrow$  alphabet.

- ①  $\phi$  is a regular language
- ② For each  $a \in \Sigma$ ,  $\{a\}$  is a regular language
- ③ If  $L_1, L_2, \dots, L_n$  are regular languages and where  $n$  is any natural number then,  $L_1 \cup L_2 \cup L_3 \dots \cup L_n$  is regular lang.
- ④ If  $L_1, L_2, \dots, L_n$  are regular languages then  $\bigcup_{i=1}^n L_i$  is regular language
- ⑤ If  $L$  is a regular language then  $L^*$  is a regular language
- ⑥ Nothing <sup>else</sup> is a regular language unless it is constructed using point 1 to 5.

## Standard Representation of Regular Languages

24.2.23 - Fri

- ① Regular Expression
- ② ~~DFA~~ DFA (Deterministic Finite Automata)
- ③ NFA (Non-deterministic Finite Automata)
- ④ Regular Grammar

## ① Regular Expression

A regular expression is a string that describes the whole set of strings according to a certain syntax rule.

(a) In string closed parenthesis '(' is a regular expression

(b)  $x \in \Sigma$  <sup>← alphabet</sup>

(c) If  $R$  are regular expressions then  $(R)^*$  is a RE and  $R^+$  is also RE.

(d) If  $R_1, R_2$  are RE then  $(R_1 | R_2)$   $(R_1 \cdot R_2)$

## Component of Regular Expression

+  $\rightarrow$  concatenation union (or)

•  $\rightarrow$  concatenation (and)

\*  $\rightarrow$  Kleen Star

(Ex) Regular Expression containing even no. of zeroes

$$RE = \emptyset(00)^*$$

$$= \{ \epsilon, 00, 0000, \dots \}$$

\* From regular set we can construct regular language.

(Ex) Regular expression that generates odd no. of 1's

$$RE = \{ (10)^* 1 \}$$



Ex Regular expression that generates any no. of 0 followed by any no. 1 but starts with 00.

$$RE = \{ (00)^* (0+1)^* \}$$

Ex RE to identify a multiline comment in C.

Ex RE for 'for' 'while' 'do while' loop.

Ex RE to generate a string with even no. of a's followed by odd no. of b's

$$RE = \{ (aa)^* (bb)^* b \}$$

Ex RE to generate a string with any no. of 0's and 1's and having at least 1 pair of consecutive 0's

$$RE = \{ (0+1)^* (00)^* (0+1)^* \}$$

Ex RE to generate a substring aba. Alphabet = {a, b}

$$RE = \{ (a+b)^* aba (a+b)^* \}$$

\* Concept of RE are implemented by Regex.

0 → concatenation

0 (a+b) } not regular, starts with concatenation  
0 (a\* b\*)

- Q RE for language starting with a and having any combination of b.  $(2T + 92) = 1(T + 2)$
- $$RE = \{ab^*a\}$$

- Q Starting with a w/o consecutive ab

$$RE = \{(a+ab)^*\}$$

- Q Describe language :-  $(b^*(aaa)^*b^*)^*$

Language consisting of strings where a occur in triplets and there is no restriction on occurrence of b.

- Q RE for string that do not contain substring "oi".

$$RE = \{0^*1^*\}$$

$$RE = 1^*0^*$$

## Q Algebra of Regular Expressions

R, S, T are any arbitrary regular expressions.

- (a)  $(R+S)+T = R+(S+T) \Rightarrow$  (Associative Property)

(b)  $R+R = R$

(c)  $R+\phi = \phi + R = R$

(d)  $R+S = S+R \Rightarrow$  (Commutative Property)

(e)  $R\phi = \phi R = \phi$  }  $\phi = \text{null set}$   
 $\uparrow \text{and } \epsilon$  }  $\epsilon = \text{null string}$

(f)  $R.\epsilon = \epsilon.R = R$

(g)  $(RS)T = R(ST) \Rightarrow$  (Associative Property)

Jan 2



$$(h) R(S+T) = RS + RT \quad \text{(Distributive Property)}$$

$$(i) (S+T)R = (SR + TR)$$

$$(j) \phi^* = \epsilon^* = \epsilon$$

$$(k) R^* \cdot R^* = R^*$$

$$(l) R \cdot R^* = R^* \cdot R \quad \text{(Commutative)}$$

$$(m) (R+S)^* = (R^*S^*)^*$$

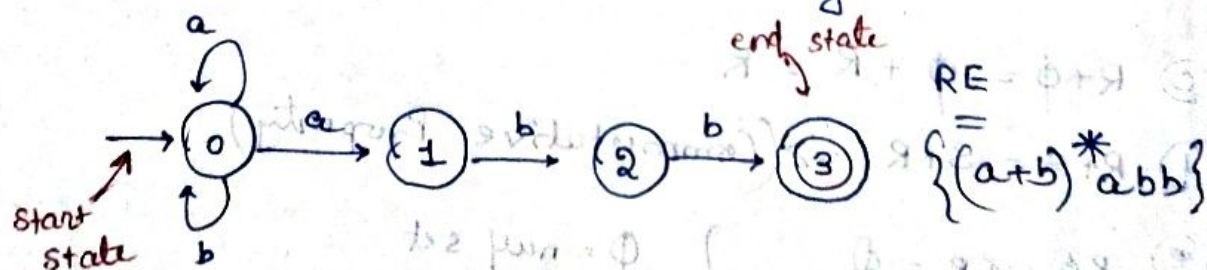
## Finite Automata

It is recognizable for a language  $L$  and is a program that takes as input a string ' $x$ ' and answers 'yes'. If  $x$  is a sentence of ' $L$ ' and it is a no otherwise and it is a part of lexical analyzer for compiler design and no otherwise.

\* From regular expression we can construct NFA and then DFA or direct DFA.

\* From DFA we can generate regular expression.

\* Ardenne Theorem can be used to design DFA from regular expression directly.



DFA  $\rightarrow$  Deterministic Finite Automata

NFA  $\rightarrow$  Non-deterministic Finite Automata

$1^+ = 1, 11, 111 \dots$  (no  $\epsilon$ )  $1^* = \epsilon, 1, 11, 111 \dots$

$0^+ 1^+ 2^+ \rightarrow$  any string using  $0, 1, 2$  excluding  $\epsilon$

$0^* 1 0^* 1 0^*$   
string contains exactly 2 1s

Q RE for all combination of a

$$RE = \{a^*\}$$

\* Read/Practise from Javatpoint

Q RE starts with 1 ends with 0

$$\{1(0+1)^*0\}$$

Q RE for string starting with a but not having consecutive b.

~~$$RE = a^*(ab)^*a^*$$~~

$$RE = \{a^*b a^*b a^*\}$$

$$RE = (a + bab)^*$$

Q Write RE for language except accepting all combination of a except num.

$$RE = \{a^+\}$$

Q RE accepting all combination accept of a or b

$$RE = \{(a+b)^*\}$$

Q RE accepting all strings starting with 1 and ending with 0

$$RE = \{1^*(0+1)^*0\}$$

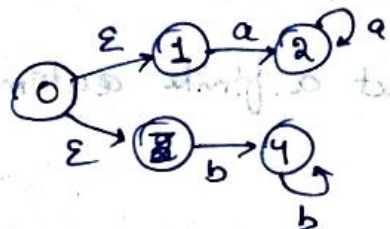


\* In NFA, ~~multiple states marks the ending~~

\* In DFA, ~~a single state is marks the ending~~

# Each diagram must have 1 start state and can have multiple ending state.

## Non-deterministic Finite Automata



\* In NFA, for same input two states are reached, so it is non-deterministic.

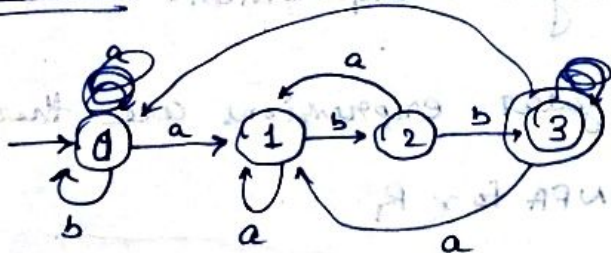
## Deterministic Finite Automata

- It has no transition on input = 'ε'
- For each state 's' and input symbol 'A' there is atmost one edge labelled 'A' leaving 's'

Ex)  $RE = \{ (a/b)^* abb \}$

DFA

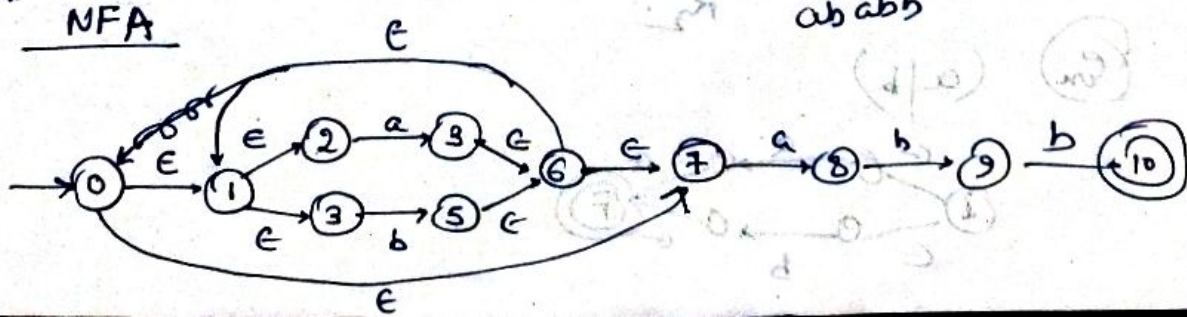
~~NFA~~



## Thompson Set Construction Method

\* According to this we can construct NFA

\* NFA



## Algorithm

Input is our regular expression (R) over alphabet ( $\Sigma$ )  
and output is NFA  $\rightarrow$  accepting lang. denoted by R

## Method

$(a|b)^*abb$

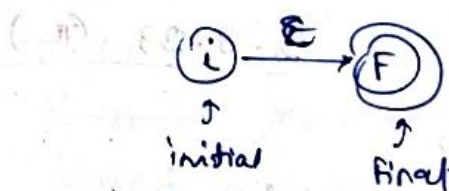
① Decompose the regular expression into primitive component.

② For each component, construct a finite automaton inductively as follows.



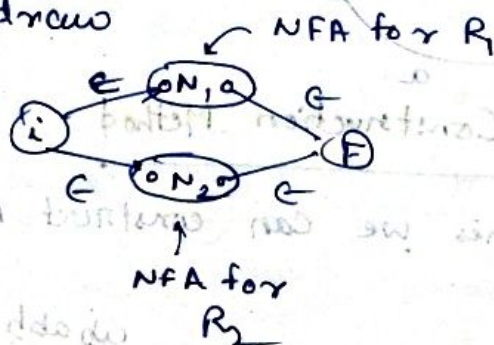
### Basis

① If 'ε' is there, how to construct

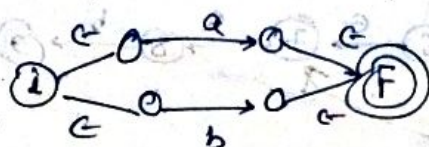


③ Having constructed components for the basis, regular expression, proceed to combine them to form the complex regular expression.

①  $R_1 | R_2$  type of regular expressions are there then draw



$(a|b)$





(ii)  $R_1 R_2$  type (concatenation)



(En) ~~a~~ a.b

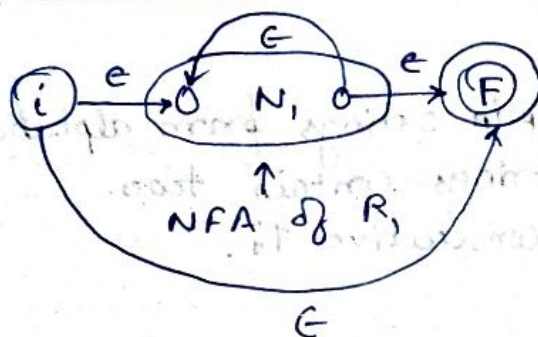


\* initial and final state of both/all NFA's should be merged.

(iii) ( )  $\rightarrow$  bracket is same NFA

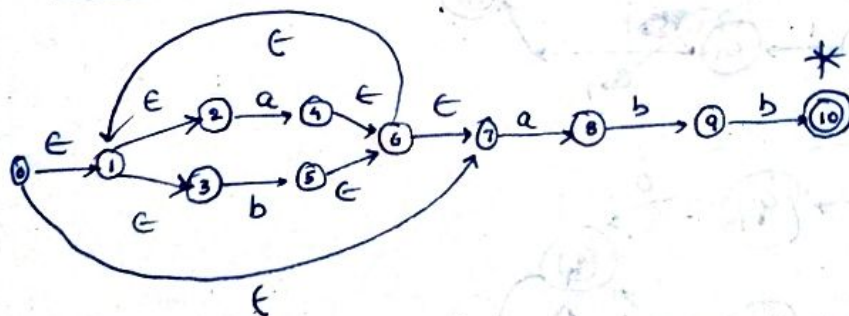


(iv)  $R_1^*$  Type

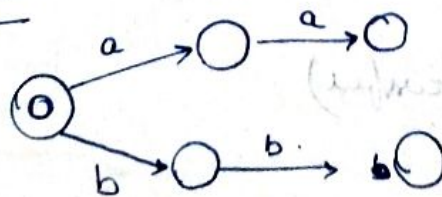


3.3.23. (fm)

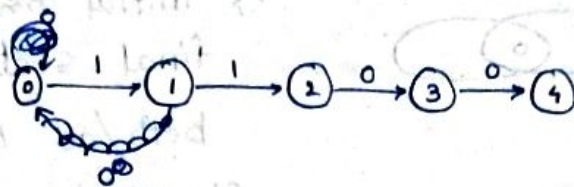
$(a|b)^*abb$



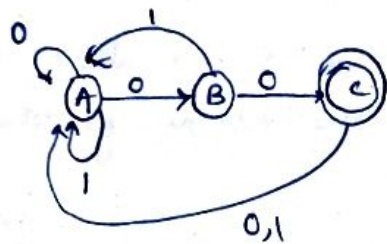
$aa^*|bb^*$



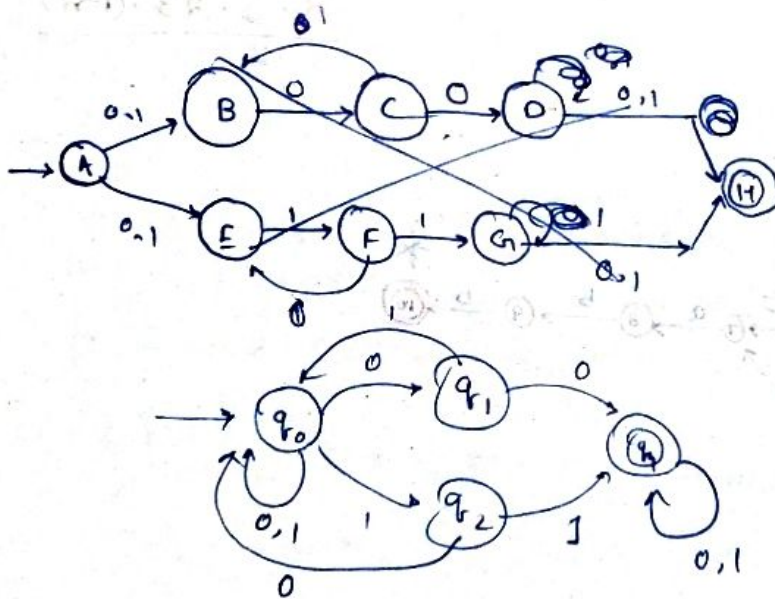
Q Design a NFA which accepts string '100' only.



Q Design a NFA to accept set of strings over  $\Sigma = \{0,1\}$  and ending with 2 consecutive zeros.



Q Design a NFA to accept strings from alphabet  $\{0,1\}$  such that the strings contain two consecutive 0's or two consecutive 1's.



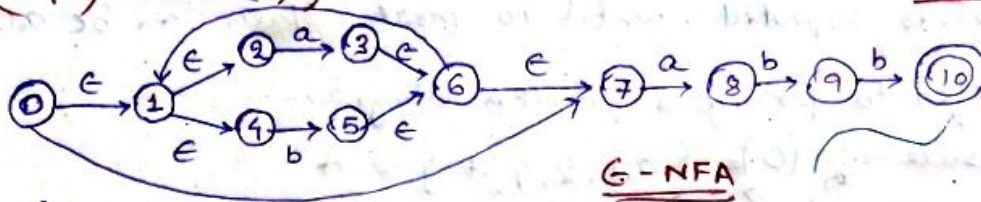
Text  $\rightarrow$  10100 (successful)



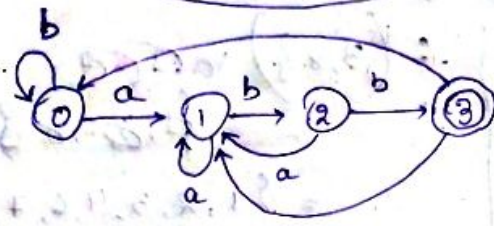
# Constructing a Deterministic Finite Automata from NFA.

$(a/b)^*abb$

16.3.23 - (Thu)



G-NFA



DFA

\* In DFA no. of transition = no. of VP and there must not be duplicacy

\* In DFA,  $\epsilon$  transitions are invalid.

## Method of Converting NFA to DFA

- Each state of DFA (D) is a set of states which NFA (N) would be after reading some sequence of inputs.
- D is able to simulate in parallel all possible moves NFA can make on a given I/P string.
- The initial state of DFA is the set consisting of  $S_0$  (start state) initial state of NFA together all states of NFA that can be reached from  $S_0$  by means of  $\epsilon$  transitions only.
- The accepting states (final states) of D are those sets of states that contain at least one accepting state of N.
- Define  $\epsilon$ -closure(s). How to calculate?  
→  $\epsilon$ -closure = What are the states that can be reached from a specific state by ' $\epsilon$ ' input only.
- (a) s is added to  $\epsilon$ -closure(s)
- (b) If t is in  $\epsilon$ -closure(s) and there is an edge labelled 'c' from 't' to 'u' then 'u' is added to  $\epsilon$ -closure.



if it is not already there.

Step (b) is repeated until no more states can be added.

\* Refer to previous CNFA diagram.

$$\text{E-closure } \phi(0) = \{0, 1, 2, 4, 7\} = A$$

or

$$E^+(0)$$

$$A^a = \{3, 8\}$$

↓

States that can be reached from elements of A when input is 'a'

$$A^b = \{5\}$$

$$E^+(\{3, 8\}) = \{1, 2, 4, 7\} \cup \{1, 2, 4, 7\} = \{1, 2, 3, 4, 6, 7, 8\} = B$$

$$E^+(\{5\}) = \{5, 6, 7, 1, 2, 4\} = \{1, 2, 4, 5, 6, 7\} = C$$

Check for a and b I/P for all sets

$$B^a = \{3, 8\} \text{ (already calculated) not need to calculate again}$$

$$B^b = \{5, 9\}$$

$$E^+(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\} = D$$

over  
X X X X  
X

$$C^a = \{3, 8\} \text{ (already calc.)}$$

$$C^b = \{5, 9\} \text{ (already calc.)}$$

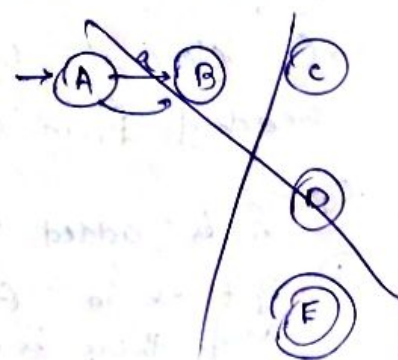
$$D^a = \{3, 8\} = C$$

$$D^b = \{5, 10\} \quad E^+(\{5, 10\}) = \{1, 2, 4, 5, 6, 7, 10\} = E$$

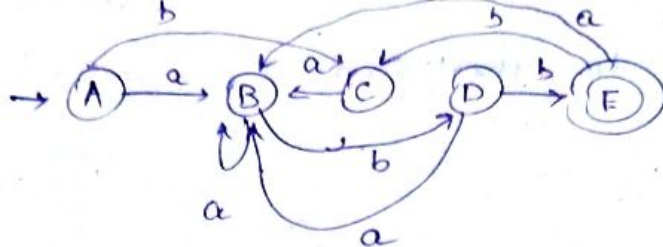
$$E^a = \{3, 8\} = B$$

$$E^b = \{5\} = C$$

| Transition Table |   |   |
|------------------|---|---|
| State            | a | b |
| → A              | B | C |
| B                | B | D |
| C                | B | C |
| D                | B | E |
| * E              | B | C |







24.3.23 - (Fri)

Minimization of DFA - Minimizing  
no. of states of DFA

| State | Inputs |   |
|-------|--------|---|
|       | a      | b |
| → A   | B      | C |
| B     | B      | D |
| C     | B      | C |
| D     | B      | E |
| * E   | B      | C |

Input

Input  $\mathcal{D}$  A DFA  $m$  with sets of states ( $S$ ) with inputs ( $\Sigma$ )

Initial state ( $S_0$ )

Set of final states ( $F$ )

Output

A DFA  $m'$  accepting same language as  $m$  and having as few states as possible.

- 1)  $\mathcal{D}$  We construct a partition ( $\pi$ ) of the set of states. Initially ( $\pi$ ) consists of 2 groups  
→ final states and non-final states

(ABCD) (E)

- 2) Group with E can't be splitted  
so we split (ABCD)

$$ABCD]^a = B$$

$$ABCD]^b = \mathcal{D}$$

Since D is going to other grp  
separate D

$(ABC) (D) (E)$

$$ABC]^a = au B$$

$ABC]^b$  since B is going to D, split

$(Ac) (B) (D) (E)$

$Ac]^a = au B$  (other group)  $\rightarrow$  take representative among A and C

$Ac]^b = au C$  (same grp) let's take 'A' (representative)

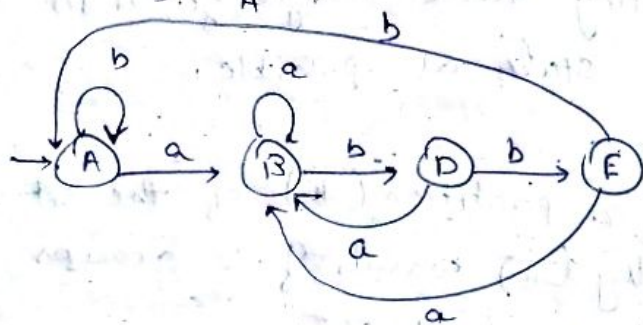
so wherever there is C  
put A

New Table

| State           | Inputs |   |
|-----------------|--------|---|
|                 | a      | b |
| $\rightarrow A$ | B      | A |
| B               | B      | D |
| C               | B      | A |
| D               | B      | E |
| * E             | B      | A |

Redundant

reject it



match with

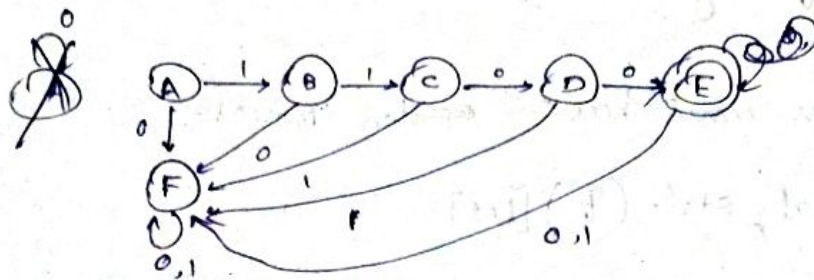
Q Design a DFA which accepts set of all strings  
contains 1100 as substring where  $\Sigma = \{0,1\}$

$$RE \rightarrow (0+1)^* 1100 (0+1)^*$$



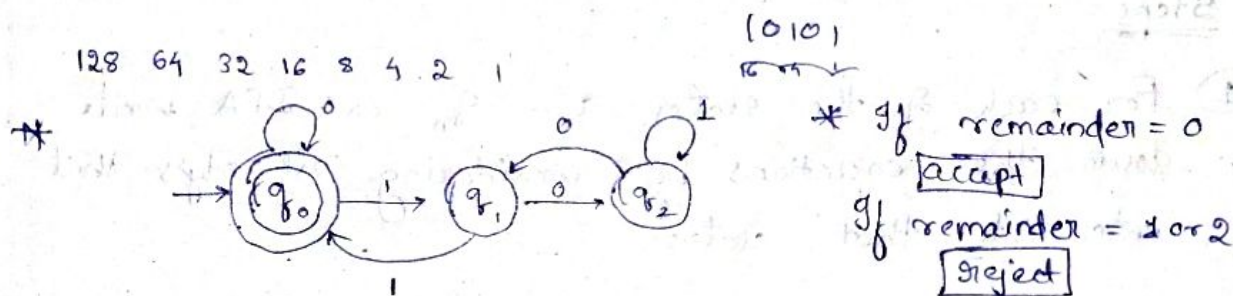


Q Design DFA accepting only 1100

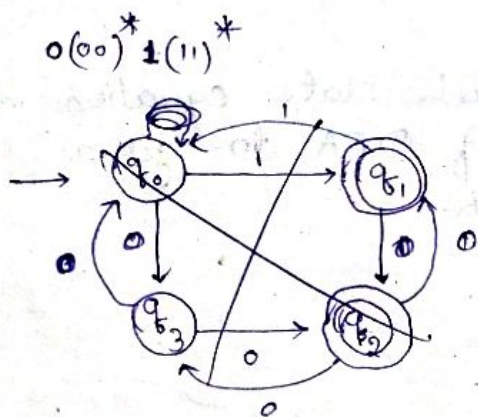


11.4.23 - (Tue Mon)

Q Design a DFA which accepts set of all strings containing  $\cdot \cdot \cdot 3 = 0$  where string is treated as binary numbers.

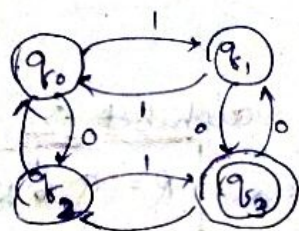
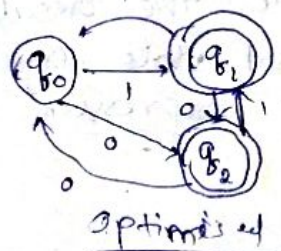


Q Design a DFA which accepts set of all strings containing odd no. of 0's and odd no. of 1's.



Possibilities

01, 0001, 0111, 000111, 101010, ...



# Construction of Regular Expression

## Assumptions

- ① The diagrams must have epsilon moves.
- ② It has <sup>an</sup> initial state ( $q_1$ ) [let]
- ③ It has vertices ( $q_1, q_2, \dots, q_n$ )
- ④  $q_n$  is the RE representing the set of strings accepted by the system through  $q_n$ , i.e. the final step.

## Steps

- ① For each of the states  $q_1, \dots, q_n$  in DFA write down the equations by considering all edges that enter into that state.
- ② For the initial state of DFA, the equation is added with epsilon ( $\epsilon$ ).
- ③ Compute the equation for each state.
- ④ Substitute the result of each state equation into the final state equation of DFA to get a regular expression of DFA.

## Arden's Theorem

Arden's Theorem suggests,  
Let P and Q are two RE over alphabet  $\Sigma$  and if P doesn't contain epsilon ( $\epsilon$ ) then  $R = QP^*$  has unique sol'n

$$\begin{aligned} \text{If } Q = \epsilon \\ R &= (\epsilon)P^* \\ &= \epsilon P^* \end{aligned}$$

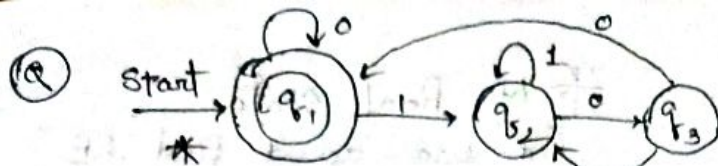
$$\begin{aligned} \text{If } Q = \epsilon \\ (\text{origin}) \\ R &= Q + RP \\ &= \epsilon + RP \\ &= \epsilon P^* \\ &= P^* \end{aligned}$$

$$R = Q^+ RP$$

Then this

If this





$$r_1 = r_1(0) + r_3(0) + \epsilon \leftarrow \text{Initial state} \quad (1)$$

$$r_2 = r_2(1) + r_1(1) + r_3(1) \quad (2)$$

$$r_3 = r_2(0) \quad (3)$$

Substituting (3) in (2)

$$r_2 = r_2(1) + r_1(1) + r_2(0)$$

$$\left. \begin{aligned} r_2 &= \frac{r_2}{R} \left( \frac{1+01}{P} \right) + \frac{r_1(1)}{Q} \end{aligned} \right\} \text{Arden's Theorem}$$

$$r_2 = r_1(1+01)^* \quad (4)$$

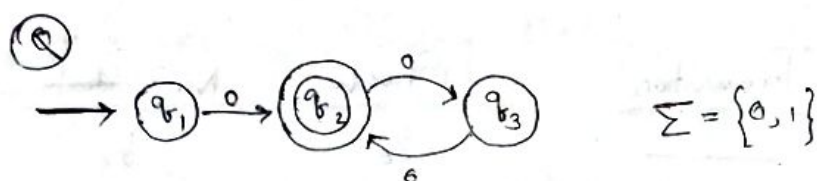
Substituting (4) in (1)

$$r_1 = r_1(0) + r_2(0) + \epsilon \quad (5)$$

$$r_1 = 0(r_1 + r_2) + \epsilon$$

Substituting (4) in (5)

$$r_1 = r_1(0) + r_1(1+01)^*0 + \epsilon$$



19.4.28 - (Wed)

$$r_1 = r_1(0) + \epsilon \quad (1)$$

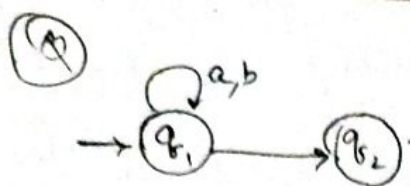
$$r_2 = r_1(0) + r_3(0) \quad (2)$$

$$r_3 = r_2(0) \quad (3)$$

Substituting (3) in (2)

$$r_2 = r_2(0) + r_1(0)$$

$$r_2 = r_1(0)^* = \epsilon \cdot 0(00)^*$$



$q_1 = q_1 a + q_1 b \rightarrow 1$

\* No final state  
So we can't find RE  
So  $RE = \emptyset$

\*  $\epsilon \rightarrow$  no character in string

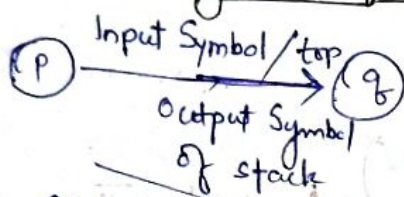
\*  $\emptyset \rightarrow$  all  $\epsilon$ s in a set

## PDA (Push Down Automata)

PDA = FSM + Stack

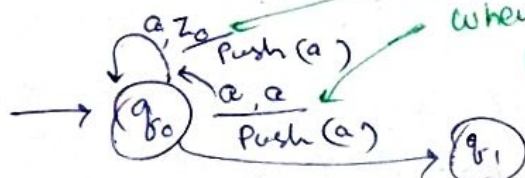
$\rightarrow$  accepts context free language (for from memory)

## Transition diagram of PDA



$p \rightarrow q$   
Push(p)

operations of stack



if i/p is a and top is  $z_0$  push 'a'  
when a is input and top is also a push 'a'

top = a  
i/p = b  
then pop

\* Since no final state, string may not be accepted.

## Transition Table of PDA

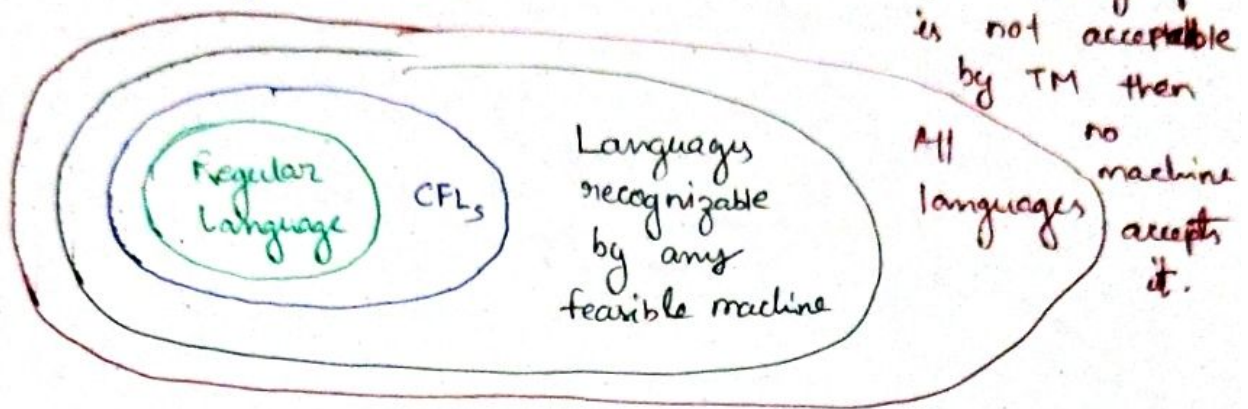
| Unread i/p | Transition                         | Stack    | New state |
|------------|------------------------------------|----------|-----------|
| aab        |                                    | $z_0$    | $q_0$     |
| ab         | $(q_0, a, z_0, \text{push}(a))$    | $az_0$   | $q_0$     |
| b          | $(q_0, a, a, \text{push}(a), q_0)$ | $aa z_0$ | $q_0$     |
| $\epsilon$ | $(q_0, b, a, \text{pop}(b), q_1)$  | $az_0$   | $q_1$     |



# Turing Machine \*NOTES IN DIGITAL PPT\*

25.4.22 (Tue)

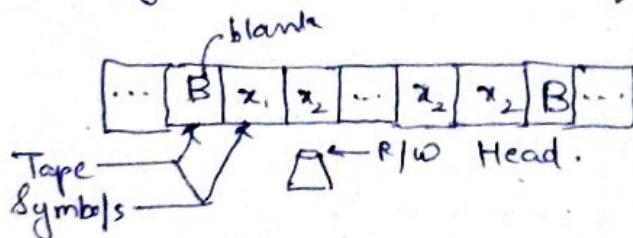
- Accepts all languages
- Language accepted by Turing Machine is called recursively enumerable.



## Church-Turing's Thesis

Any mathematical problem solving that can be described by an algorithm can be modeled by a Turing Machine.

- Turing machine consists of an infinitely long tape, in both directions.



## Steps of Turing Machine

- ① Start in  $q_0$
- ② Read symbol under head
- ③ Write new symbol.
- ④ Shift left/right.
- ⑤ Enter new state  $q_j$

